Automatic 3D Terrain Construction System Based on ROS and MoveIt

Project Team

June 23, 2026

Contents

1 Project Overview and Task Goals 2

1.1 Operating Environment 2

1.2 Actuator 2

1.3 Core Tasks 3

2 System Design 3

2.1 Driver Layer 4

2.2 Recognition Layer 5

2.3 Control Layer 5

3 Recognition Layer 5

3.1 Input and Output 6

3.2 Color-Height Mapping 6

3.3 Core Processing Function 6

3.4 Technical Workflow 7

4 Control Layer 8

4.1 Class Initialization 8

4.2 Control Workflow 8

4.2.1 Initialization Phase 8

4.2.2 Robot Arm Motion Workflow and Position Control 9

4.2.3 Block Motion and Block Management 11

4.2.4 High-Frequency Block Tracking via TF2 11

5 Experimental Results 14

6 Conclusion 15

Appendix: Team Roles 15

1 Project Overview and Task Goals

This project builds an automated robotic system based on ROS Noetic and MoveIt for automatic 3D terrain construction.

1.1 Operating Environment

- ROS Noetic + MoveIt + RViz

1.2 Actuator

- Franka Emika Panda 7-joint robot arm

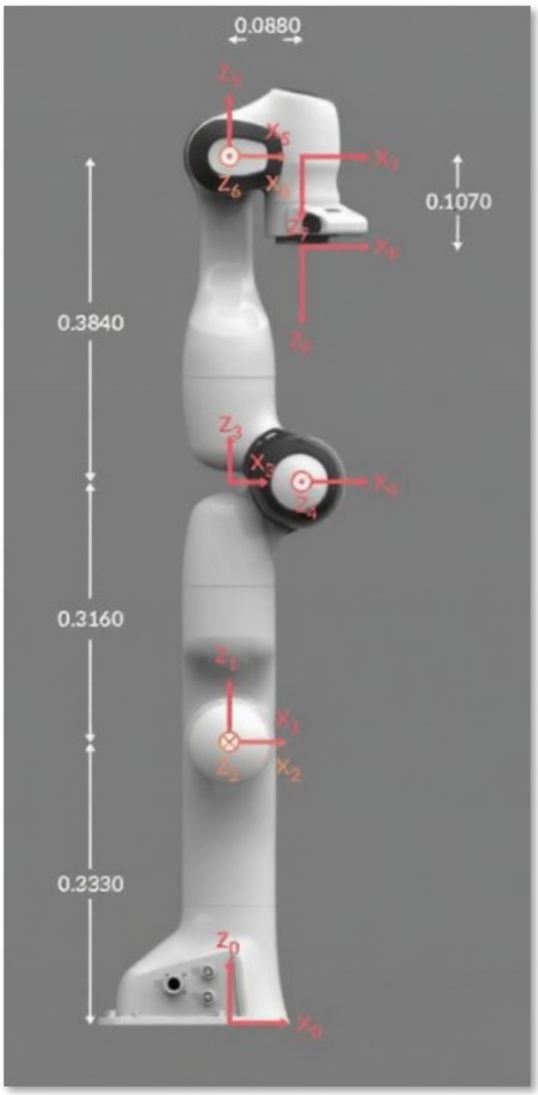


Figure 1: Franka Emika Panda robot arm structure

Joint	a (m)	d (m)	α (rad)	θ (rad)
Joint 1	0	0.333	0	θ_1
Joint 2	0	0	$-\frac{\pi}{2}$	θ_2
Joint 3	0	0.316	$\frac{\pi}{2}$	θ_3
Joint 4	0.0825	0	$\frac{\pi}{2}$	θ_4
Joint 5	-0.0825	0.384	$-\frac{\pi}{2}$	θ_5
Joint 6	0	0	$\frac{\pi}{2}$	θ_6
Joint 7	0.088	0	$\frac{\pi}{2}$	θ_7
Flange	0	0.107	0	0

Figure 2: Denavit-Hartenberg (D-H) parameters of the Panda arm

The Franka Emika Panda is a 7-degree-of-freedom anthropomorphic robot arm. **Figure 1** shows the physical structure and joint layout, while **Figure 2** presents the corresponding D-H parameter table, which defines the kinematic relationships between successive links. These parameters are essential for forward and inverse kinematics computations in the MoveIt planning pipeline.

1.3 Core Tasks

- Simple contour to 32×32 matrix
- Maximum stacking height: 8 layers
- Hundreds of high-precision pick-and-place cycles
- Imitate real terrain

2 System Design

The system adopts a three-layer architecture: Driver Layer, Recognition Layer, and Control Layer.

2.1 Driver Layer

The Driver Layer is based on ROS + MoveIt + RViz and relies on the `panda_moveit` package, responsible for robot arm motion control and visualization.

The launch file `terrain_builder_demo.launch` launches the entire system with the following configuration:

- Robot arm type: Franka Emika Panda
- Mode: Simulation
- Controller: OMPL → RRT + collision detection

terrain_builder_demo.launch

```
<!-- Launch Panda + MoveIt core -->
<include file="$(find panda_moveit_config)/launch/demo.launch">
  <arg name="arm_id" value="panda" />
  <arg name="pipeline" value="ompl" />
  <arg name="moveit_controller_manager" value="fake" />
  <arg name="fake_execution_type" value="interpolate" />
</include>

<!-- Launch RViz with terrain-optimized config -->
<node name="rviz" pkg="rviz" type="rviz"
      args="-d terrain_builder.rviz" />

<!-- Run terrain builder after initialization -->
<node name="terrain_builder"
      pkg="panda_moveit_config"
      type="terrain_builder.py"
      output="screen"
      launch-prefix="bash -c 'sleep 3; exec $0 $@'" />
```

This launch file accomplishes three key tasks:

1. Load the Panda robot arm URDF model and the Open Motion Planning Library (OMPL) kinematics solver
2. Launch RViz with a visualization configuration optimized for terrain building
3. Start the core control node `terrain_builder.py` after a 3-second delay

2.2 Recognition Layer

The core module of the Recognition Layer is `contour_to_heightmap.py`, whose main functions include:

- Convert simple contour maps to matrices
- Generate a 32×32 terrain matrix

2.3 Control Layer

The core module of the Control Layer is `terrain_builder.py`, whose main functions include:

- Launch file startup
- Robot arm control
- Block control

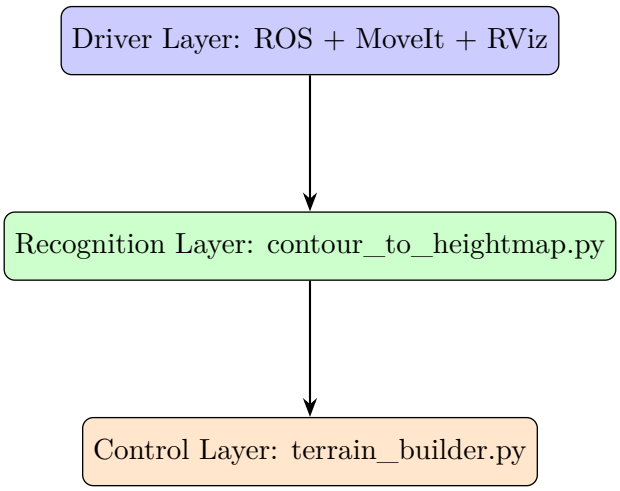


Figure 3: System Three-Layer Architecture

3 Recognition Layer

The Recognition Layer is responsible for converting contour terrain maps into height matrices that the robot arm can recognize.

3.1 Input and Output

- **Input**: Colored contour map (colors represent heights)
- **Processing**: Grid sampling + color-height matching
- **Output**: 32×32 matrix with max element up to 8

3.2 Color-Height Mapping

The system predefines 8 contour levels with corresponding colors:

contour_to_heightmap.py: Color Mapping

```
CONTOUR_COLORS_BGR = {
    1: (255, 0, 0),       # Blue
    2: (0, 255, 0),       # Green
    3: (0, 255, 255),     # Yellow
    4: (0, 165, 255),     # Orange
    5: (0, 0, 255),       # Red
    6: (255, 0, 255),     # Magenta
    7: (255, 255, 0),     # Cyan
    8: (128, 0, 128),     # Purple
}
```

3.3 Core Processing Function

The conversion from image to height matrix is implemented by the `image_to_heightmap` function:

contour_to_heightmap.py: Image to Height Matrix

```
def image_to_heightmap(image, grid_size=8, num_levels=8):
    """Convert a contour image to a 2D heightmap matrix."""
    h, w = image.shape[:2]
    heightmap = np.zeros((grid_size, grid_size), dtype=np.int32)
    cell_w = float(w) / grid_size
    cell_h = float(h) / grid_size

    for i in range(grid_size):
        for j in range(grid_size):
            # Sample the pixel at the center of each grid cell
```

```
            px = int((j + 0.5) * cell_w)
            py = int((i + 0.5) * cell_h)
            bgr = tuple(image[py, px])
            level = _color_to_level(bgr)
            if level > 0:
                heightmap[i, j] = level
    return heightmap
```

This function divides the image into an $N \times N$ grid, samples the center pixel of each cell, and determines the height level through color matching.

3.4 Technical Workflow

1. Import contour map
2. Pixel sampling
3. Color-height mapping
4. Generate `heightmap.npy`

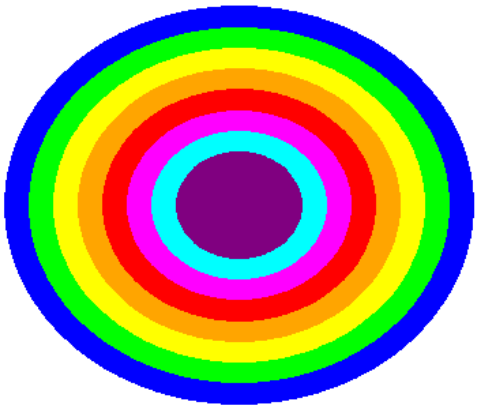


Figure 4: Input: Colored contour map

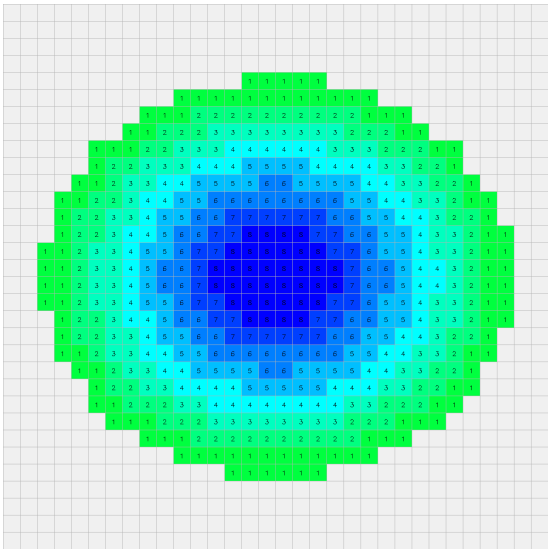


Figure 5: Output: 32×32 height matrix visualization

4 Control Layer

The Control Layer is responsible for robot arm motion planning and block management. The core class is `TerrainBuilder`, implemented in `terrain_builder.py`.

4.1 Class Initialization

terrain_builder.py: Core Class Initialization

```
class TerrainBuilder(object):
    def __init__(self, grid_size=8, num_levels=8):
        # MoveIt interface initialization
        self.robot = moveit_commander.RobotCommander()
        self.scene = moveit_commander.PlanningSceneInterface()
        self.move_group = moveit_commander.MoveGroupCommander("
            panda_arm")

        # Terrain parameters
        self.grid_size = grid_size          # 32×32
        self.num_levels = num_levels        # Max 8 layers
        self.block_size = 0.02              # Block side length 2cm
        self.grid_spacing = 0.02            # Grid spacing 2cm
        self.approach_height = 0.10         # Safe approach height

        # State tracking
        self.heightmap = None
        self.current_heights = None
        self.blocks_placed = 0
        self.total_blocks = 0
```

4.2 Control Workflow

After the `terrain_builder.py` node initializes, it executes the following steps sequentially:

4.2.1 Initialization Phase

- Variable settings: block, grid

- Recognition module call: `contour_to_heightmap.py`
- Related node communication
- Visualization settings

4.2.2 Robot Arm Motion Workflow and Position Control

The robot arm motion workflow covers the complete path planning and position control from picking to placing:

1. Up the block
2. Block generating
3. Down to grasp position
4. Close the gripper
5. Pick up
6. Move with block
7. 14 cm above destination
8. Loose the gripper

The core pick function `pick_from_supply` is implemented as:

terrain_builder.py: Pick Block

```
def pick_from_supply(self):
    # Move above the supply area
    pre_z = self.supply_z + self.approach_height
    self.center_block_above(self.supply_x, self.supply_y, pre_z)

    # Open gripper, descend precisely
    self.open_gripper()
    grasp_flange_z = self.supply_z - self._z_offset * self._grasp_zz
    self.cartesian_move_to_z(grasp_flange_z)

    # Close gripper to grasp block
    self.close_gripper()

```

```
    # Create marker that follows the gripper
    self._start_tracking(held_marker)

    # Remove collision object from supply area
    self.scene.remove_world_object(supply_name)

    # Lift up and return
    self.cartesian_move(self.approach_height + 0.05)
    return supply_name
```

The system achieves precise pick-and-place operations through position control, ensuring each block reaches its target location accurately. The core position control function `center_block_above` implements precise IK-based motion:

terrain_builder.py: Position Control

```
def center_block_above(self, cx, cy, z_target):
    # Calculate flange target position (align block center to target
        point)
    flange_x = cx - self._z_offset * self._grasp_zx
    flange_y = cy - self._z_offset * self._grasp_zy
    flange_z = z_target - self._z_offset * self._grasp_zz

    # Use full-pose IK
    q = quaternion_from_euler(self.GRASP_ROLL, self.GRASP_PITCH,
       self.GRASP_YAW)
    pose = geometry_msgs.msg.Pose()
    pose.position.x = flange_x
    pose.position.y = flange_y
    pose.position.z = flange_z
    pose.orientation = q

    self.move_group.set_pose_target(pose)
    success = self.move_group.go(wait=True)
    self.move_group.clear_pose_targets()
    return success
```

For the final 14 cm precise vertical descent, the system employs Cartesian path planning:

terrain_builder.py: Cartesian Path

```python
def cartesian_move_to_z(self, target_z):
    current = self.move_group.get_current_pose().pose
    target = copy.deepcopy(current)
    target.position.z = target_z

    # Compute Cartesian straight-line path
    (plan, fraction) = self.move_group.compute_cartesian_path(
        [target], 0.005, True)

    if plan and len(plan.joint_trajectory.points) > 0:
        return self.move_group.execute(plan, wait=True)
    return False
```

4.2.3 Block Motion and Block Management

Block Motion Workflow:

1. Generate
2. Move with arm
3. Drop with g acceleration

4.2.4 High-Frequency Block Tracking via TF2

During transport, the system must keep the visual block marker precisely aligned with the gripper in RViz. The conventional approach —calling MoveIt's `get_current_pose()` —suffers from ROS topic latency and planning scene state locks, causing visible lag and "trailing" of the block marker.

To resolve this, the system directly queries the TF2 (Transform Framework 2) tree. TF2 is ROS's core component for maintaining coordinate frame relationships.

The `robot_state_publisher` node publishes transformations between all robot links at high frequency (default 100Hz), storing them in the `tf2_buffer` for real-time queries.

The `TerrainBuilder` class initializes the TF2 buffer and listener as follows:

terrain_builder.py: TF2 Initialization

```python
# TF2 initialization — for high-frequency, low-latency transform
   lookups
```

```
self._tf_buffer = tf2_ros.Buffer()
self._tf_listener = tf2_ros.TransformListener(self._tf_buffer)
```

After a successful grasp, the system calls `_start_tracking` to launch a 50Hz timer. In the timer callback `_tracking_cb`, it queries the latest transform from the gripper frame `panda_link8` to the world frame `planning_frame` via `lookup_transform`:

terrain_builder.py: TF2 High-Frequency Tracking

```
def _start_tracking(self, marker):
    """Start tracking — marker follows the gripper via TF2 at 50Hz"""
    self._held_marker = marker
    self._block_held = True
    # 50Hz timer, querying TF every 20ms
    self._tracking_timer = rospy.Timer(
        rospy.Duration(0.02), self._tracking_cb)

def _tracking_cb(self, event):
    """Timer callback: retrieve real-time gripper pose via TF2 and update marker"""
    try:
        # Query the latest TF transform (rospy.Time(0) = latest available)
        transform = self._tf_buffer.lookup_transform(
            self.planning_frame,   # target frame (world)
            self.eef_link,         # source frame (gripper)
            rospy.Time(0))         # latest timestamp

        # Compute block center position (gripper position + fixed offset)
        q = transform.transform.rotation
        zx = 2.0 * (q.x * q.z + q.y * q.w)
        zy = 2.0 * (q.y * q.z - q.x * q.w)
        zz = 1.0 - 2.0 * (q.x * q.x + q.y * q.y)

        self._held_marker.pose.position.x = (
            transform.transform.translation.x + self._z_offset * zx)
        self._held_marker.pose.position.y = (
            transform.transform.translation.y + self._z_offset * zy)
        self._held_marker.pose.position.z = (
```

```
            transform.transform.translation.z + self._z_offset * zz)

        # Publish the updated marker
        self._held_marker_pub.publish(MarkerArray(markers=[self.
            _held_marker]))
    except tf2_ros.LookupException:
        # TF not yet available — skip this frame silently
        pass
```

The key advantage of this approach is that TF2 lookups are essentially memory reads —they involve no ROS service calls or topic communication, yielding microsecond-level latency. Combined with the 50Hz refresh rate, the block marker follows the gripper with virtually zero lag, visually "sticking" to the gripper throughout the transport phase.

When the block reaches its destination and the gripper opens, the system calls `_stop_tracking` to shut down the timer and send a DELETEALL message to remove the tracking marker, completing the visual feedback loop for each pick-and-place cycle.

The gravity animation implementation in the `place_block_at` function:

terrain_builder.py: Gravity Drop Animation

```
def _animate_fall(self, name, start_z, end_x, end_y, end_z, r, g, b)
    :
    fall_dist = start_z - end_z
    g = 9.81  # gravitational acceleration
    t_total = sqrt(2.0 * fall_dist / g)
    dt = 0.03  # 30Hz refresh rate

    for step in range(steps + 1):
        t = step * dt
        z = start_z - 0.5 * g * t * t
        # Update Marker position
        fall_marker.pose.position.z = max(z, end_z)
        self._marker_pub.publish(MarkerArray(markers=[fall_marker]))
        rospy.sleep(dt)
```

Block Management:

- Block count —`blocks_placed` increments for each block placed
- Change block ID —`_supply_counter` increments for each new block generated

The `build_terrain` function iterates layer by layer to construct the terrain, while managing block picking, placement, and counting throughout the process:

terrain_builder.py: Terrain Building Main Loop + Block Management

```
def build_terrain(self):
    max_h = int(self.heightmap.max())

    # Build layer by layer
    for layer in range(1, max_h + 1):
        # Filter cells that need blocks in this layer
        cells = [(i, j) for i in range(self.grid_size)
                 for j in range(self.grid_size)
                 if self.heightmap[i, j] >= layer]
        # Sort from back to front (ensures arm approaches from the
           rear)
        cells.sort(key=lambda ij: -ij[0])

        for i, j in cells:
            # Pick a block from the supply area
            block_name = self.pick_from_supply()
            # Place the block at the target position
            self.place_block_at(block_name, i, j, layer)
            # Block management: spawn next supply block, update
               block count and ID
            self._spawn_supply_block()
            self.blocks_placed += 1  # Block count

            # Progress output
            if self.blocks_placed % 10 == 0:
                rospy.loginfo("PROGRESS: %d/%d blocks (%.1f%%)",
                              self.blocks_placed, self.total_blocks
                                 ,
                              100.0 * self.blocks_placed / self.
                                 total_blocks)
```

5 Experimental Results

After system integration and testing, the following results were achieved:

✓ Able to recognize simple contour terrain maps

✓ Able to build a $32 \times 32 \times 8$ 3D terrain model

✓ No planning errors in hundreds of pick-and-place cycles

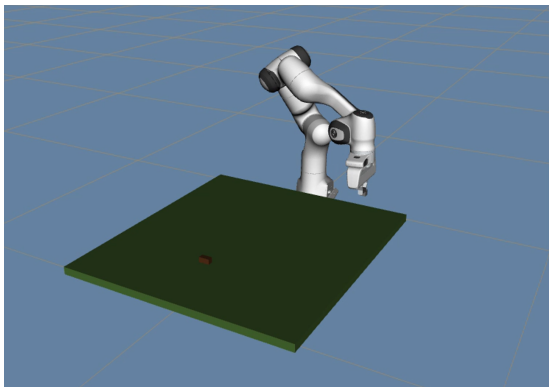


Figure 6: Building start

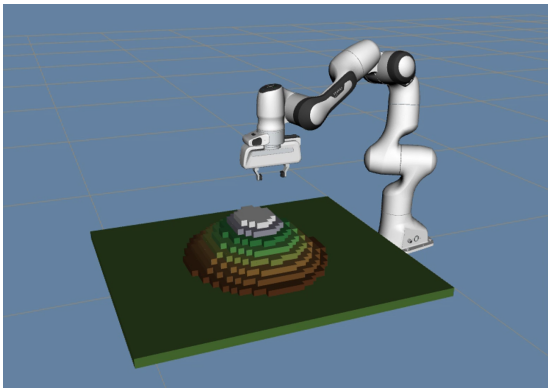


Figure 7: Building complete

6 Conclusion

This project successfully implemented an automated 3D terrain construction system based on ROS Noetic and MoveIt. Using the Franka Emika Panda 7-joint robot arm, the system can automatically convert contour terrain maps into height matrices and stack precise terrain models layer by layer.

Appendix: Team Roles

Role	**Contributions**
Software	Coding, simulation, and system integration
Documentation	Report writing and presentation
Media	Demo recording and visual assets